SERVICE-ORIENTED ARCHITECTURE

SYSTEM INTEGRATION

Arturo Mora-Rioja
amri@ek.dk

EK

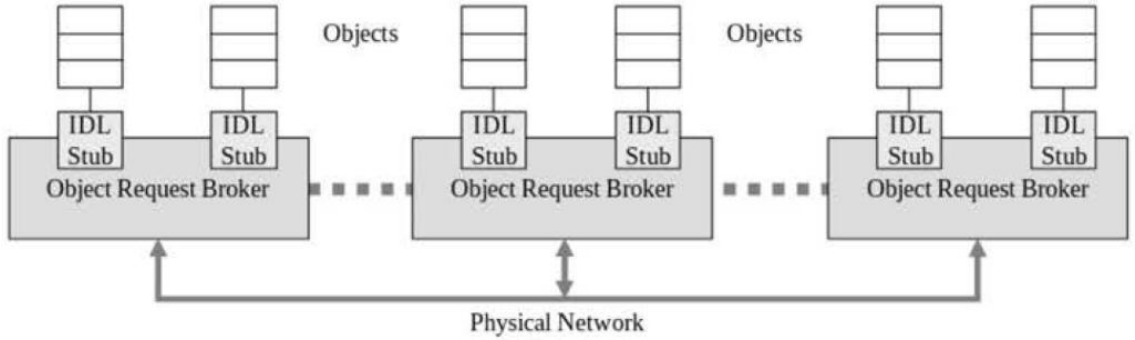
DOA – Distributed Object Architecture

- Predecessor of SOC (Service-Oriented Computing) (approx. 1990-2005)
- No distinction between clients and servers
  - All of them are objects distributed along different computers
  - Operating system- and programming language-agnostic
- Extension of OOP concepts across a network
  - Encapsulation. Only interfaces are exposed
  - Inheritance/Polymorphism. Remote interfaces can inherit from one another
- A middleware to handle communication is necessary
  - Interfaces are defined in an IDL (Interface Definition Language)
- No explicit separation of concerns
- No external mechanisms for service publication and discovery

EK

DOA – Distributed Object Architecture

- Major implementations
  - CORBA (Common Object Request Broker Architecture)
    - The middleware is a software bus called ORB (Object Request Broker)



  - Microsoft DCOM (Distributed Component Object Model)
    - Software development framework
    - Preceded by OLE (Object Linking and Embedding)
    - Precursor to Visual Studio .NET

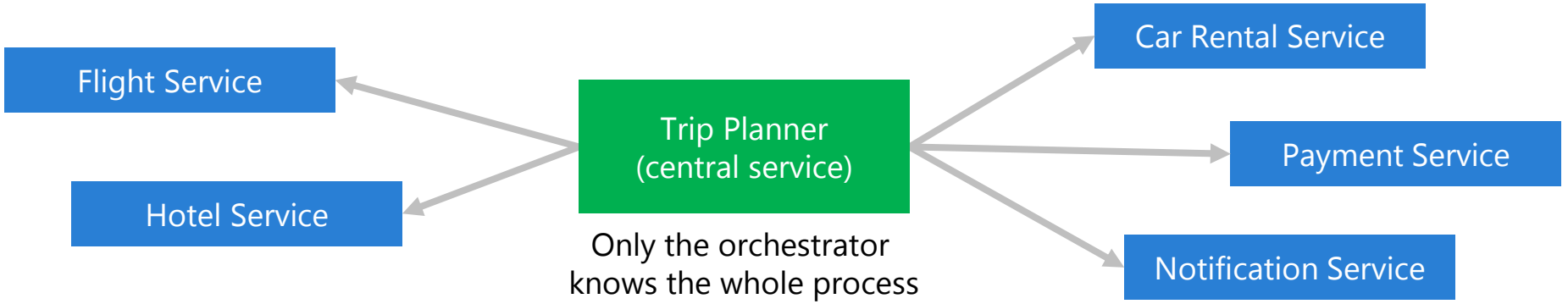
SOC – Service-Oriented Computing

- SOA – Service-Oriented Architecture: conceptual model (2000-present)
  - SOC: Design and implementation of SOA
- Service: self-contained unit of functionality
  - It can be independently designed, developed, and deployed
- Distributed services instead of distributed objects
- Explicit separation of concerns
  - The **service providers** develop services
  - The **service requesters** build the application using existing services
  - The **service brokers** publish the services and facilitate their matching and discovery
- Implementations
  - In LANs: Private SOC applications
  - In the Internet: Web service implementation

EK

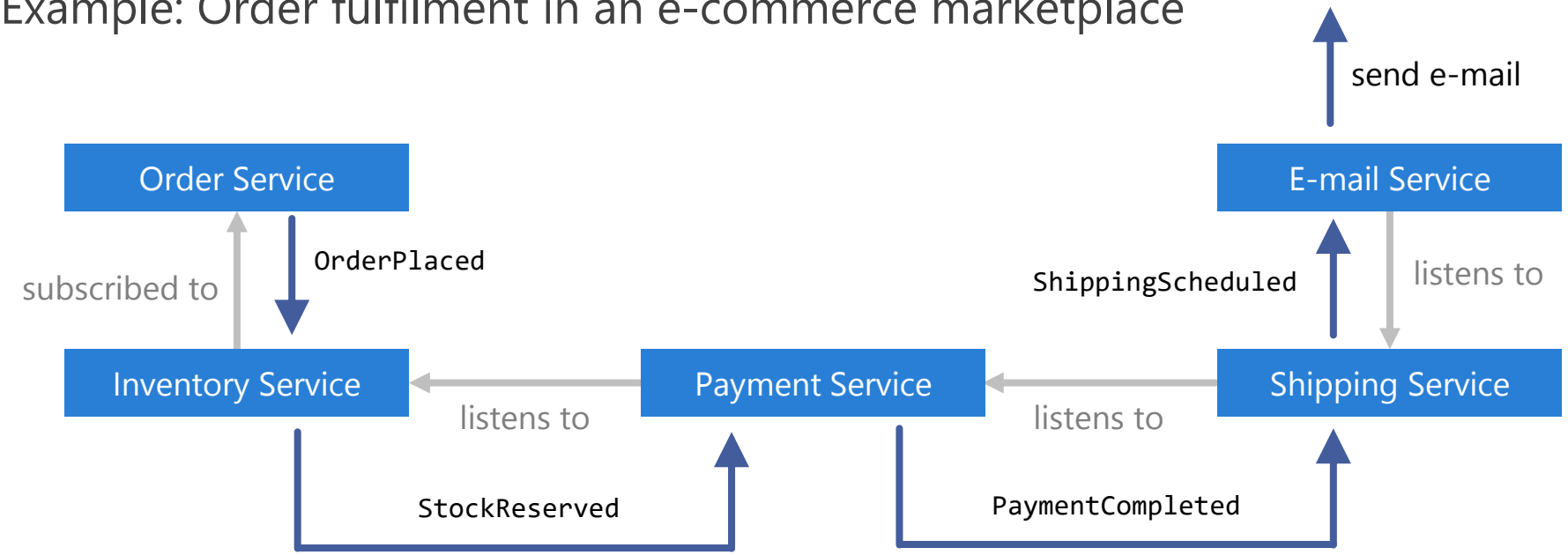
SOC – Service-Oriented Computing

- Reusable services in repositories for matching, discovery and access (either remote or local)
- Service communication through platform independent, open protocols
- **Composition**. A composite service can be created from existing services
  - **Orchestration**
    - Coordinated by a central process (which can be a service)
    - Useful for private business processes
    - Example: Online travel booking orchestrated by a Trip Planner service



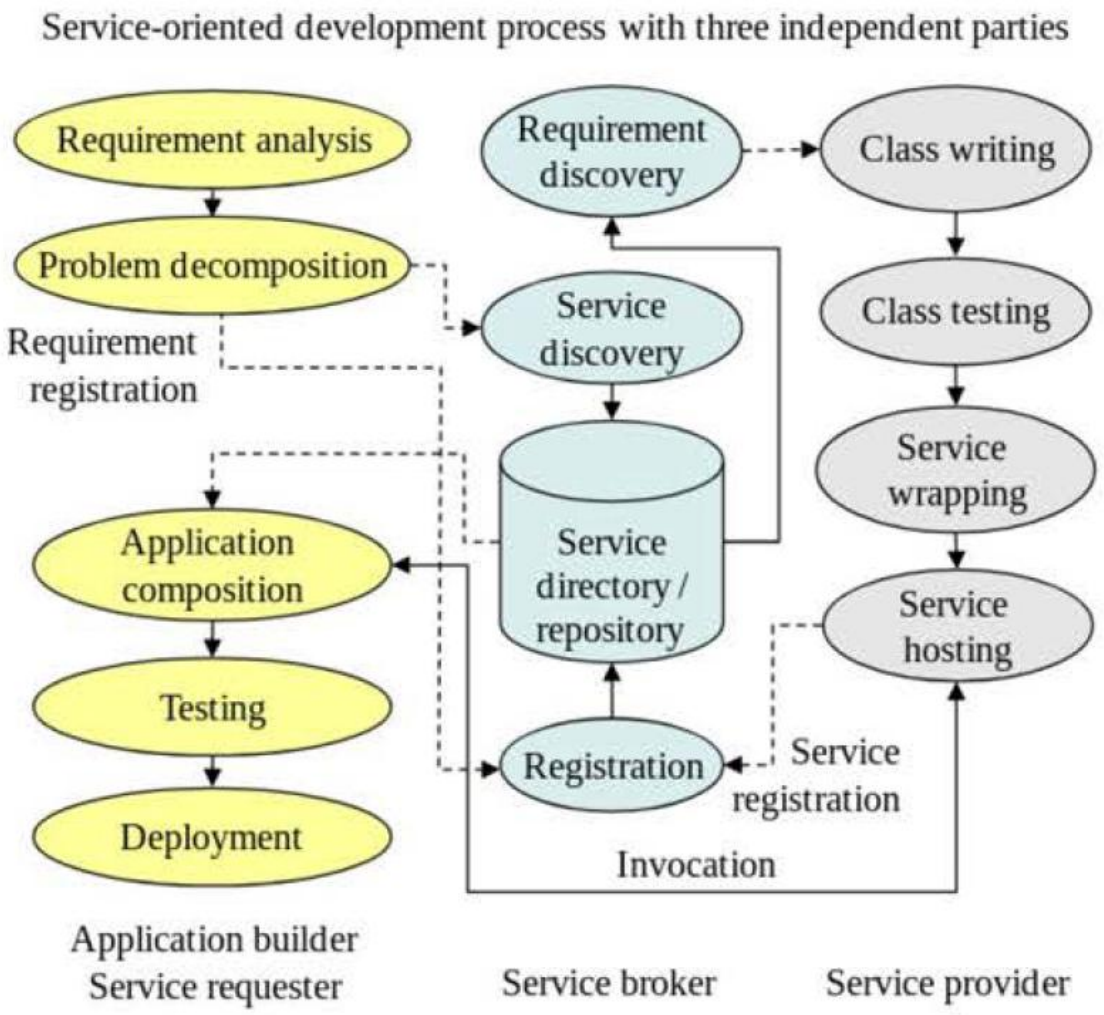
SOC – Service-Oriented Computing

- **Composition** (cont.)
  - **Choreography**
    - No central service. Services communicate among themselves
    - Useful for public business processes
    - Example: Order fulfilment in an e-commerce marketplace



SOC Services

- Atomic
  - Standard interface including:
    - The description of its functions
    - Technical details for service invocation
- Composite
  - Service composed of other services
  - Two ways of creating a composite service:
    - Static and manual. A class using classes
    - Dynamic and automatic. In production
- Services can be developed in different programming languages by different service providers and reside in different servers
- To become an SOC application, a composite service needs a human interface



SOC Features

- Self-contained and self-describing
  - Services are published through service brokers
  - They contain sufficient information for other services to discover, match, bind and invoke remotely and during production
- Reconfigurable and recomposable
  - A newly discovered service can be composed into an existing service in two ways:
    - **Reconfiguration**. An existing service is replaced by a new service that satisfies the same function specification
    - **Recomposition**. The specification of a service is changed in production. The composite service that service belongs to can incorporate new services or exclude existing ones

EK

SOC Features

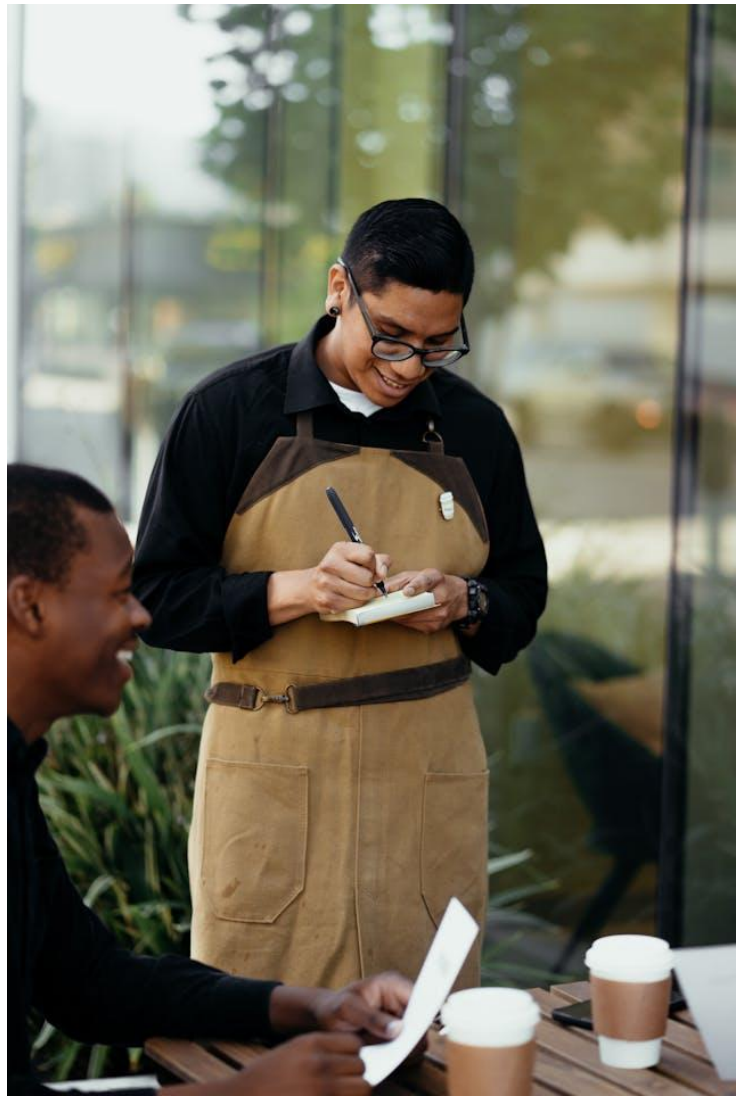
- Dynamic verification
  - Dynamically modified specifications must be dynamically verified to assure the required properties of the specification
- Dynamic validation
  - Dynamically reconfigured or recomposed services must be dynamically validated (tested) to assure that they meet the specification
- Dynamic evaluation
  - Dynamic reconfiguration or recomposition may lead to a service's structural change, so its attributes must be dynamically evaluated:
    - Reliability
    - Security
    - Safety
    - Performance

SOA – Service-Oriented Architecture

- Architectural design pattern implying a distributed software architecture
  - The software system is a collection of loosely coupled services
  - The services
    - Can reside on different computers
    - Communicate with each other
      - Through platform independent standard interfaces (e.g., WSDL)
      - Via standard message-exchange protocols (e.g., SOAP)
    - Can be reused across applications and scenarios
    - Should be discoverable
    - Can be replaced without disrupting the entire system
    - Can scale independently

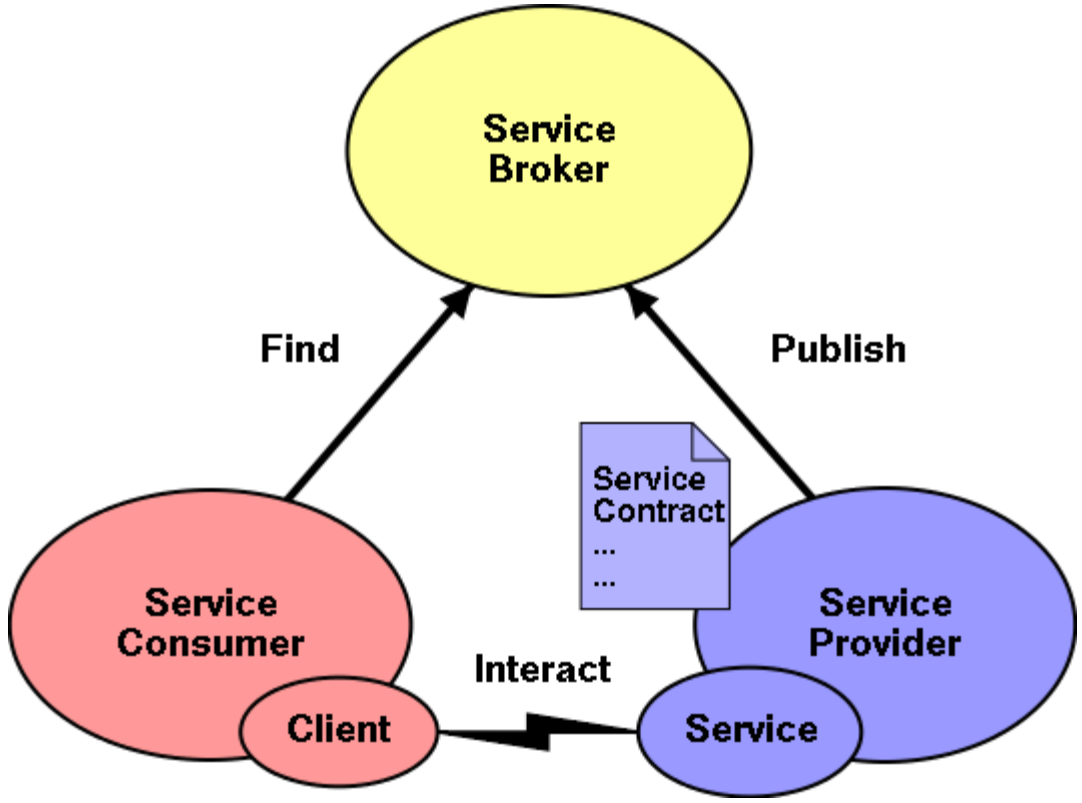
SOA – Service-Oriented Architecture

- Service
  - A function module that is well-defined, self-contained and does not depend on the context or state of other functions
  - It can be newly developed or wrap existing code
  - It normally offers an API but lacks a user interface
  - Interface between the service producer and the consumer
- Application
  - A service with a user interface



SOA – Service-Oriented Architecture

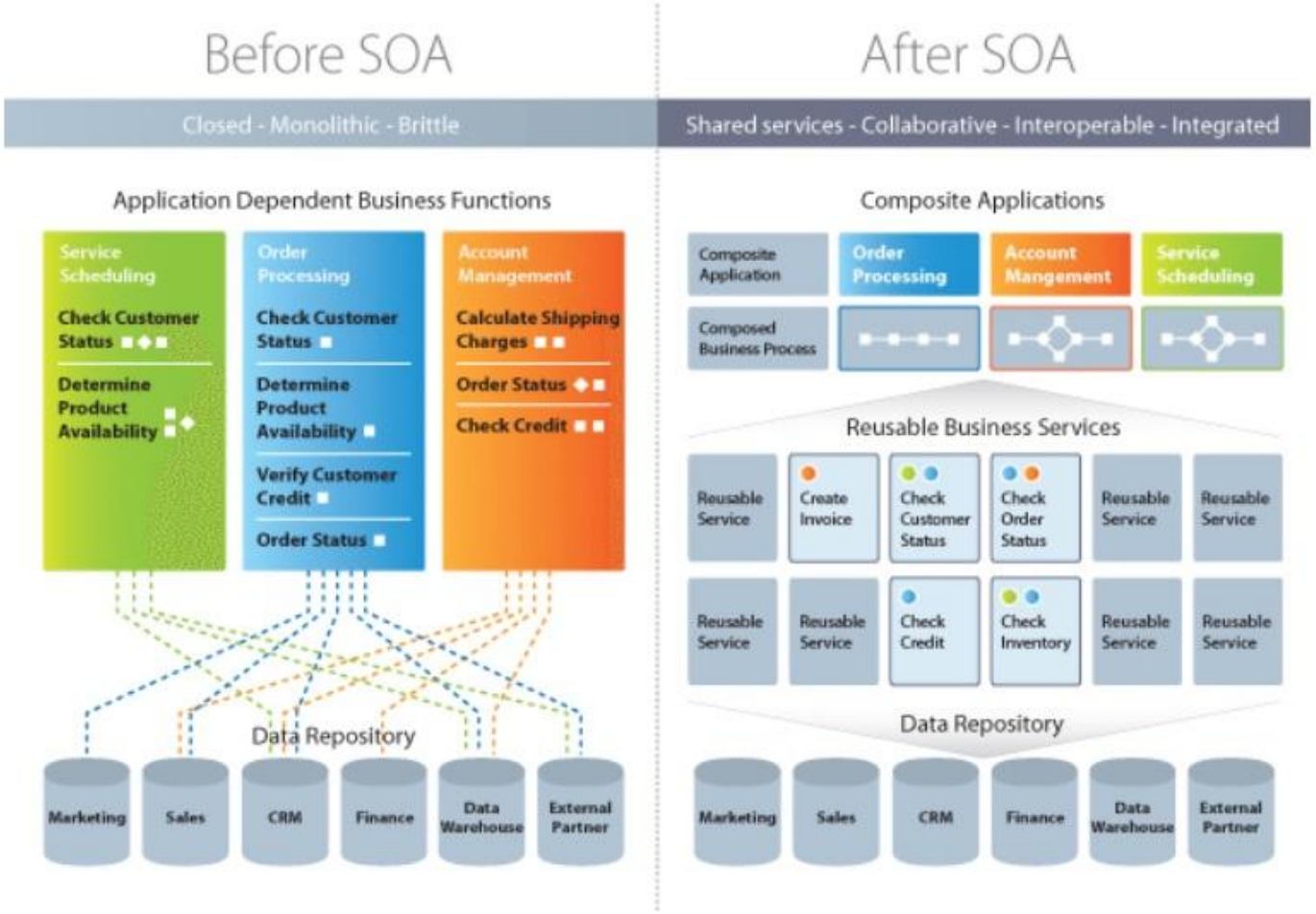
- Service Producer = Service Provider
  - Develops services
    and publishes them in service brokers
- Service Requester = Service Consumer
  - Discovers the services dynamically
    via service brokers
- Service Broker
  - Allows the producer to publish
    their service definitions and interfaces
  - Allows the consumers to search
    the database and discover services
  - Handles protocol mediation (e.g., HTTP <-> SOAP <-> MQ)
    and message transformation (e.g., XML <-> JSON <-> proprietary formats)



SOA Use Cases

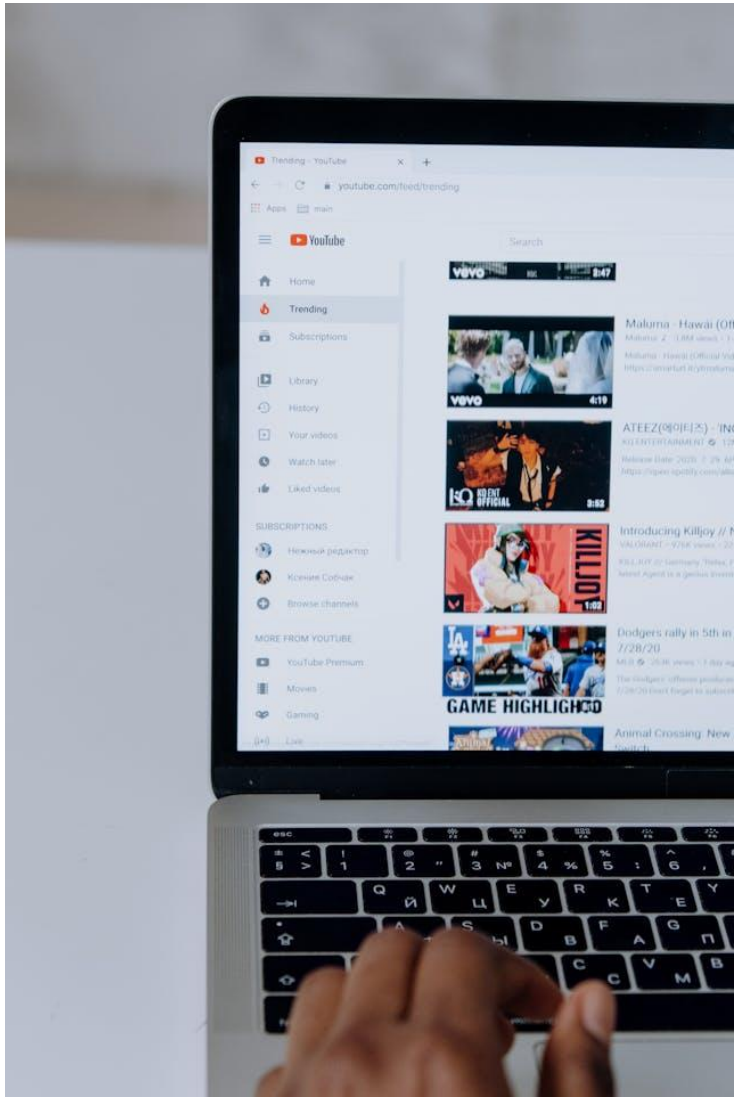
- E-commerce Integration
  - Inventory management, payment processing, shipping
- Healthcare Data Exchange
  - Patient information, medical records, lab results
- Financial Services
  - Fund transfers, currency exchange rates, account balance enquiries
- Government Services
  - Taxes, government forms, regulatory changes
- Supply Chain Management
  - Inventory, order tracking, shipment scheduling, suppliers

Monolithic vs. SOA



Web Services

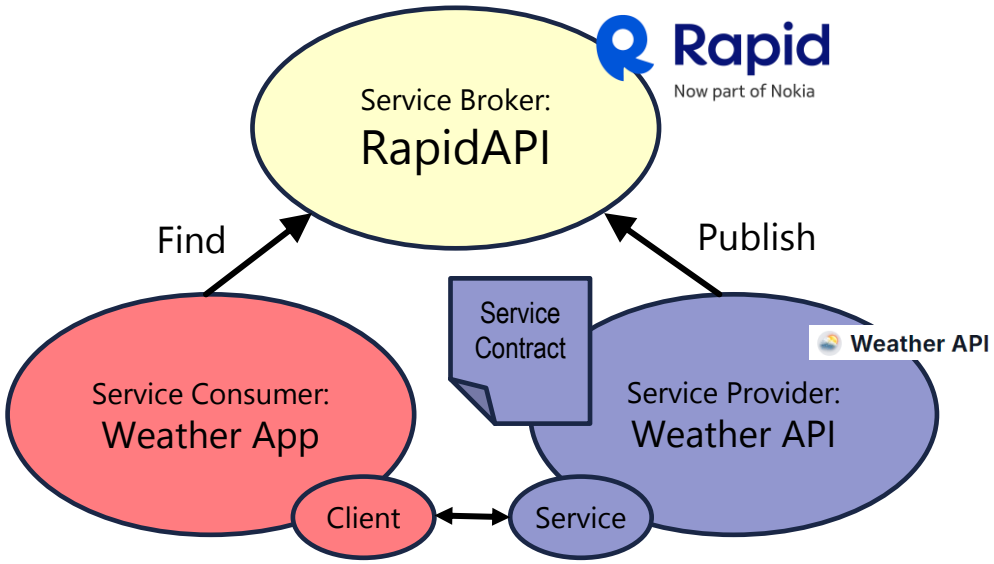
- Services accessible over the web
- A specific implementation of Service-Oriented Computing
- Identified by URL
- When a human interface is added,
  a single or composite service can become a web application
- Composite services can be created
  by communicating with other services via their interfaces
  - Loose coupling
  - Platform and programming language independence

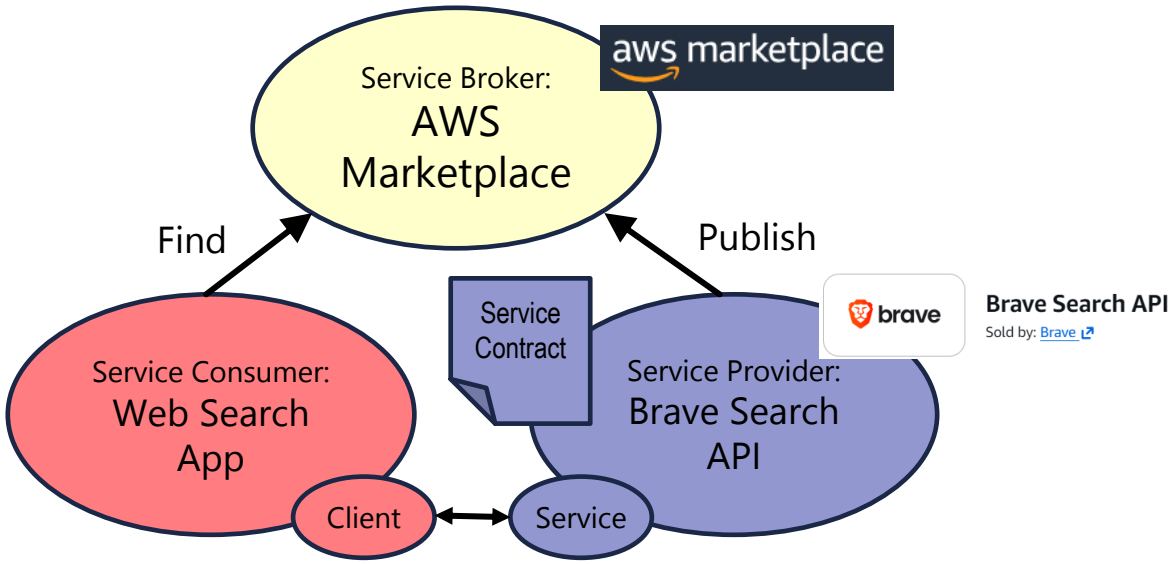


EK

Web Services Examples

- A weather application that uses the Weather API, listed in RapidAPI
- A web search application that uses the Brave Search API, listed in the AWS Marketplace





Sources

- **Chen, Yinong, and Gennaro De Luca**.
  Service-Oriented Computing and System Integration: Software, IoT, Big Data, and AI as Services, 9th ed.
  Kendall Hunt, 2024. ISBN: 979-8-3851-1757-4.
- **Das, Sourojit**.
  "Test Strategies for SOA (Service Oriented Architecture) Applications".
  BrowserStack, 2024.
- **Edet, Theophilus**.
  Web Services & Service-Oriented Architecture.
  CompreQuest, 2023.
- **Haas, Hugo**.
  "Designing the Architecture for Web Services".
  W3C, 2023.
- Cover picture by **Cytonn Photography** from Unsplash